



Kabul University

Computer Science Faculty

Information Technology Department

Computer Architecture

Memory system

CACHE MEMORY

Contents

- ✓ Computer Memory System Overview
- ✓ Characteristics of Memory Systems
- ✓ The Memory Hierarchy
- ✓ Cache Memory Principles
- ✓ Elements of Cache Design
- ✓ Cache Addresses
- ✓ Cache Size
- ✓ Mapping Function
- ✓ Replacement Algorithms
- ✓ Number of Caches

Characteristics of Memory Systems

- Internal Memory
- External Memory

Memory characteristics

Location	Performance
Internal (e.g. processor registers, main memory, cache)	Access time
External (e.g. optical disks, magnetic disks, tapes)	Cycle time
Capacity	Transfer rate
Number of words	Physical Type
Number of bytes	Semiconductor
Unit of Transfer	Magnetic
Word	Optical
Block	Magneto-optical
Access Method	Physical Characteristics
Sequential	Volatile/nonvolatile
Direct	Erasable/nonerasable
Random	Organization
Associative	Memory modules

Memory accessing methods

- **Method of accessing** units of data:

1. **Sequential access:** Memory is organized into units of data, called records. Access must be made in a specific **linear sequence**. Stored addressing information is used to separate records and assist in the retrieval process.
2. **Direct access:** As with sequential access, direct access involves a shared read/write mechanism. However, individual blocks or records have a unique address based on physical location. **Access is accomplished by direct access to reach a general neighborhood plus sequential searching, counting, or waiting to reach the final location.**

Cont.

4- Random access: Each addressable location in memory has a unique, physically wired-in addressing mechanism. The time to access a given location is independent of the sequence of prior accesses and is constant. Thus, any location can be selected at random and directly addressed and accessed. Main memory and some cache systems are random access.

5- Associative: This is a random access type of memory that enables one to make a comparison of desired bit locations within a word for a specified match, and to do this for all words simultaneously. Thus, a word is retrieved based on a portion of its contents rather than its address.

Performance

- Three performance parameter are used:
 1. **Access time (latency):** For random-access memory, this is the time it takes to perform a read or write operation, that is, the time from the instant that an address is presented to the memory to the instant that data have been stored or made available for use.
 2. **Memory cycle time:** This concept is primarily applied to random-access memory and consists of the access time plus any additional time required before a second access can begin. Note that memory cycle time is concerned with the system bus, not the processor.
 3. **Transfer rate:** This is the rate at which data can be transferred into or out of a memory unit. For random-access memory, it is equal to $1/(\text{cycle time})$.

Cont.

$$T_N = T_A + \frac{n}{R}$$

where

T_N = Average time to read or write N bits

T_A = Average access time

n = Number of bits

R = Transfer rate, in bits per second (bps)

The Memory Hierarchy

- The design constraints on a computer's memory can be summed up by three questions: How much? How fast? How expensive?
- Faster access time, greater cost per bit
- Greater capacity, smaller cost per bit
- Greater capacity, slower access time

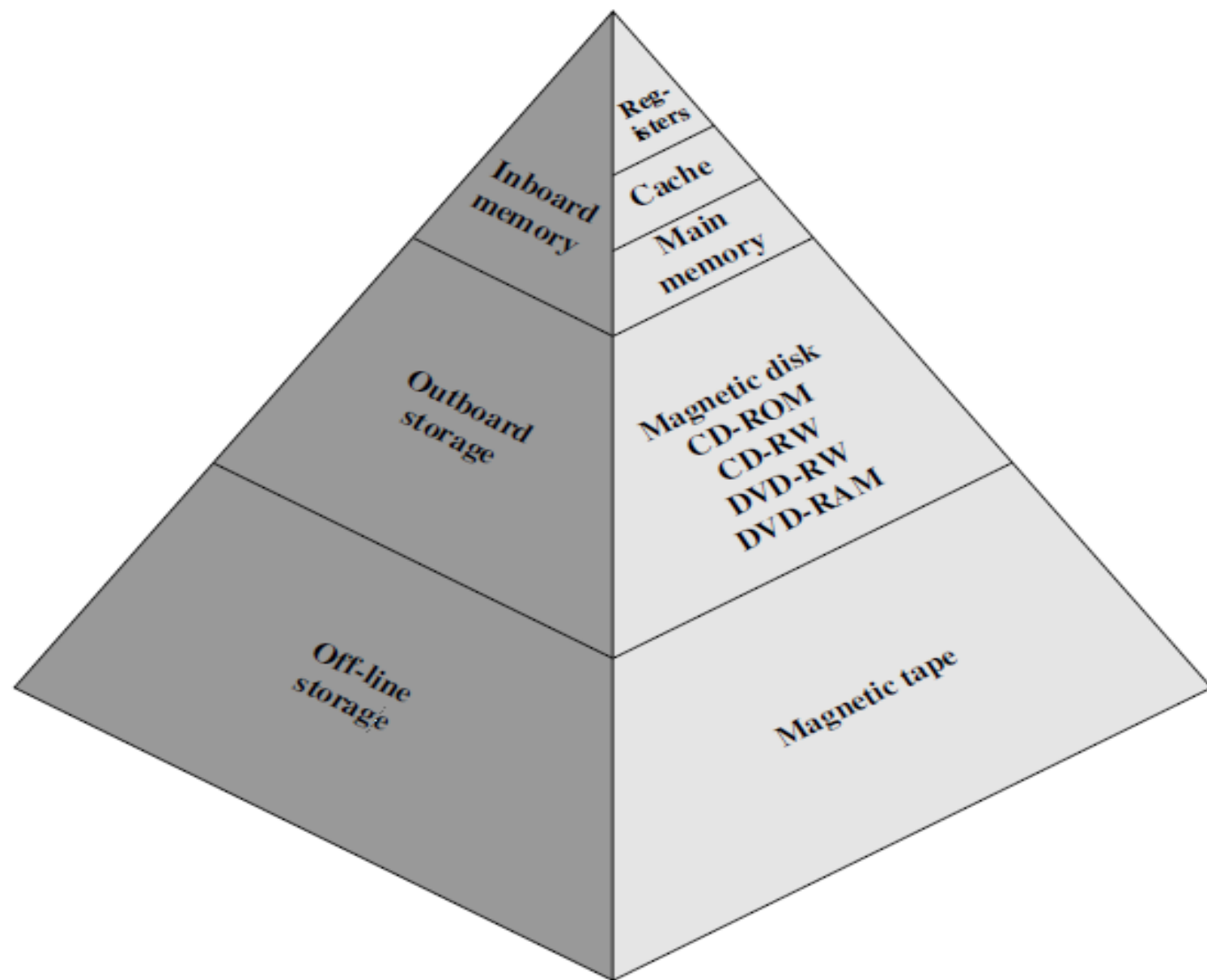
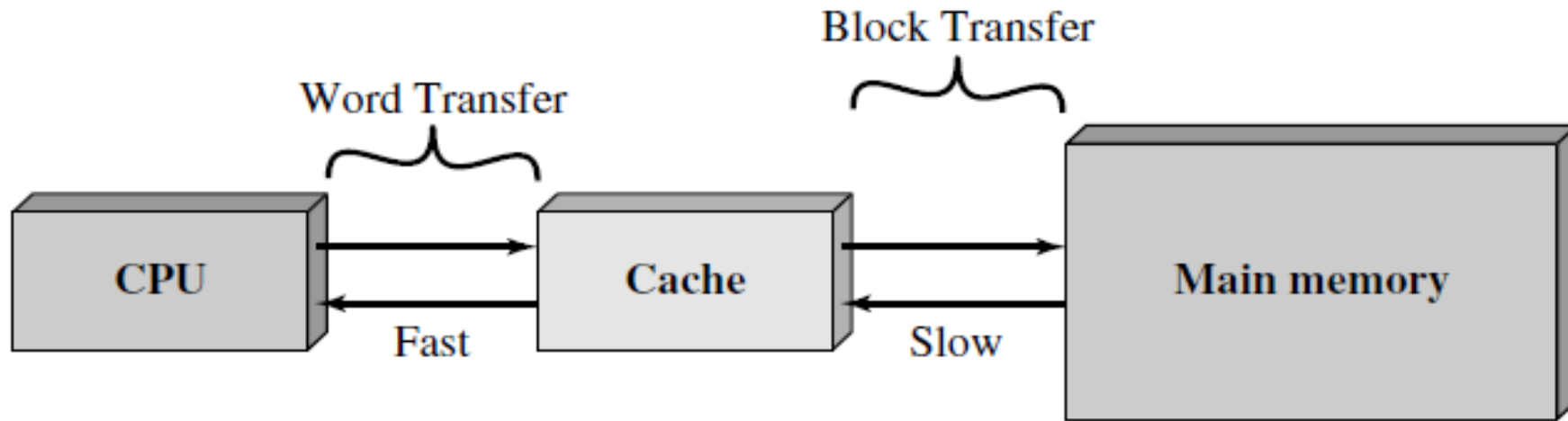


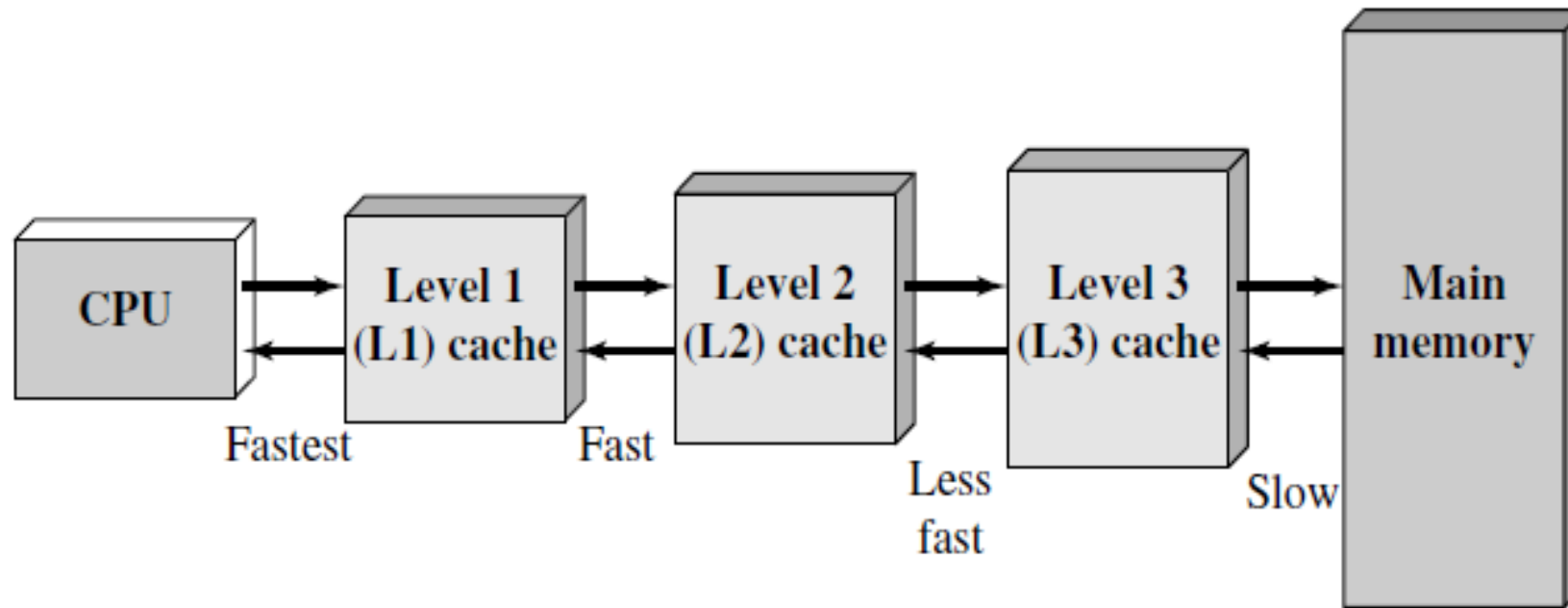
Figure 4.1 The Memory Hierarchy

CACHE MEMORY PRINCIPLES



(a) Single cache

CACHE MEMORY PRINCIPLES



(b) Three-level cache organization

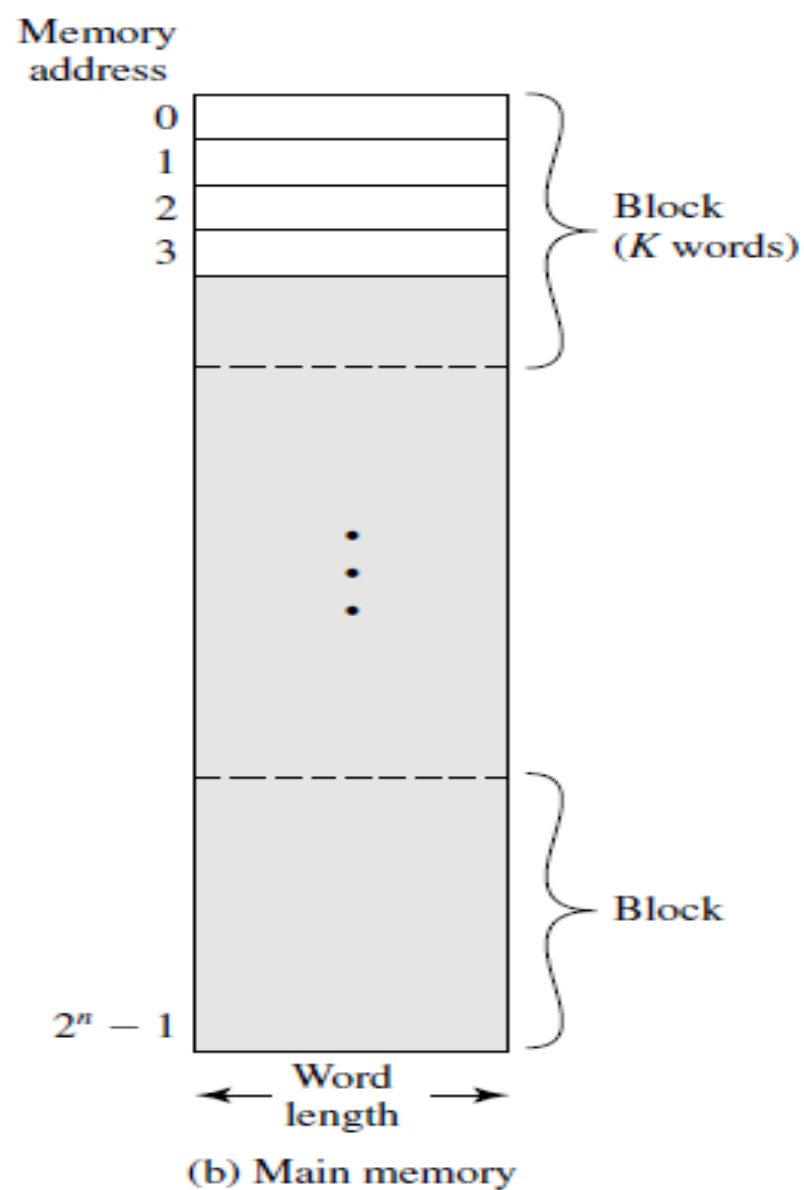
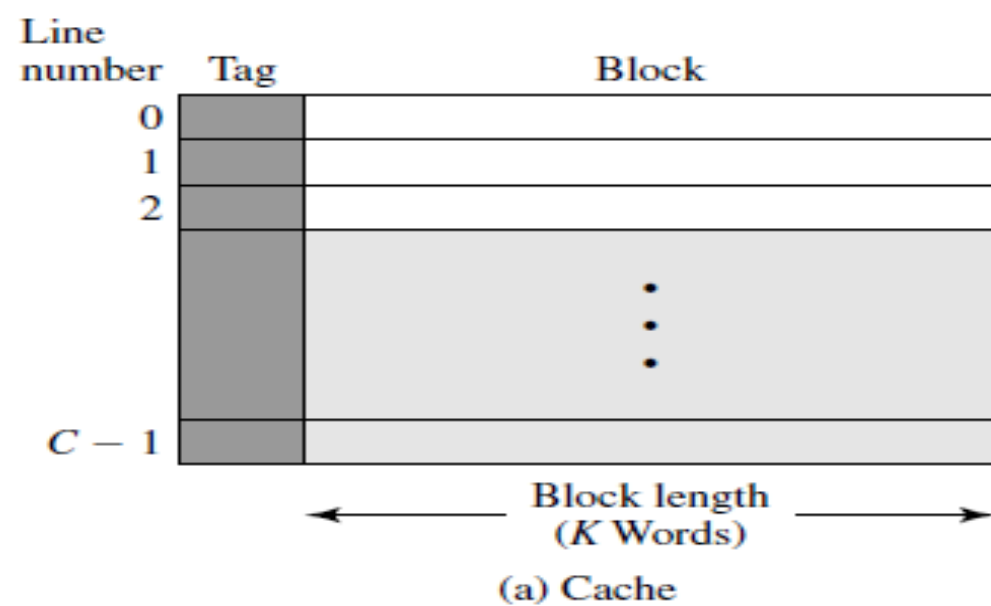


Figure 4.4 Cache/Main Memory Structure

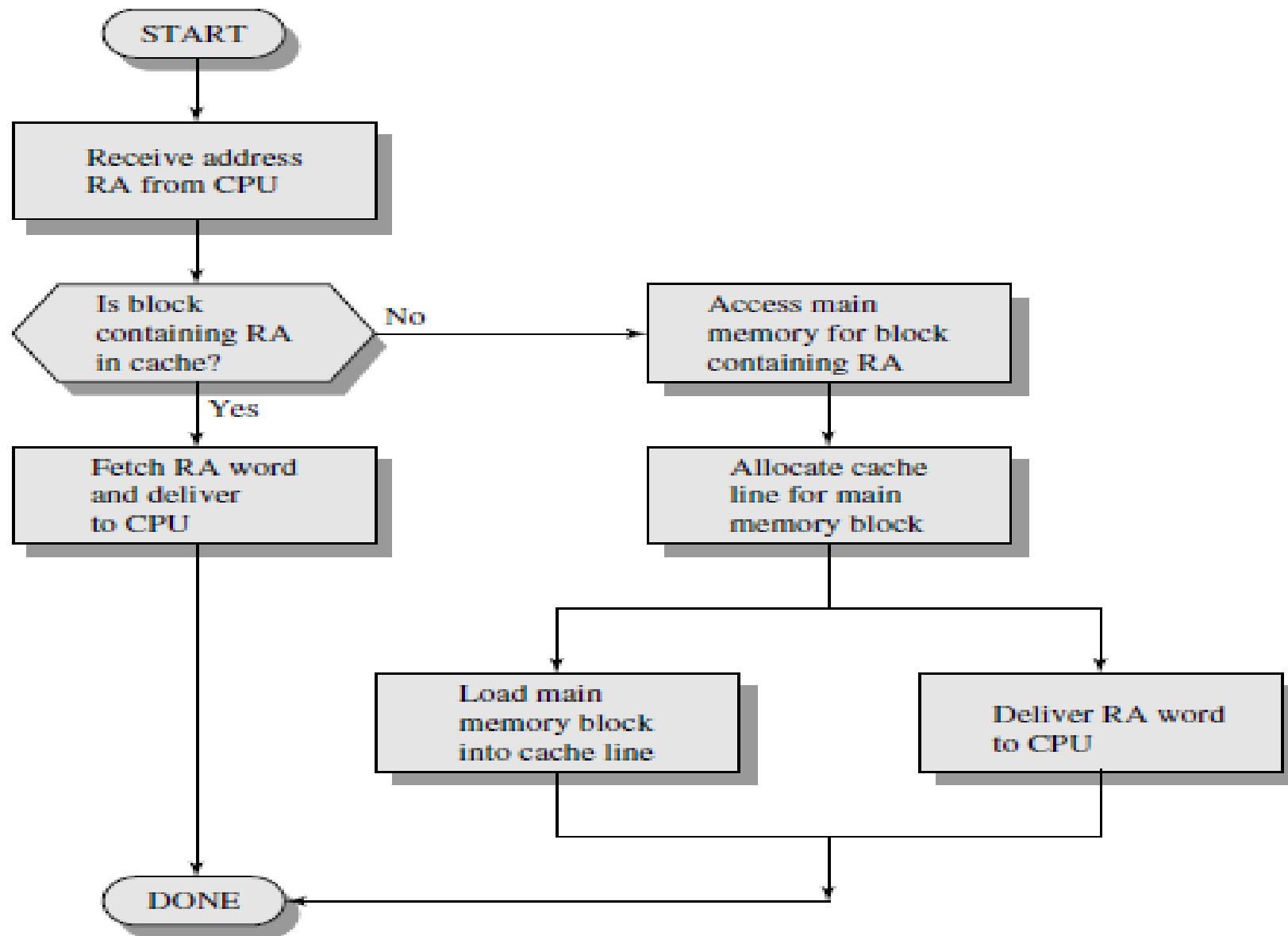


Figure 4.5 Cache Read Operation

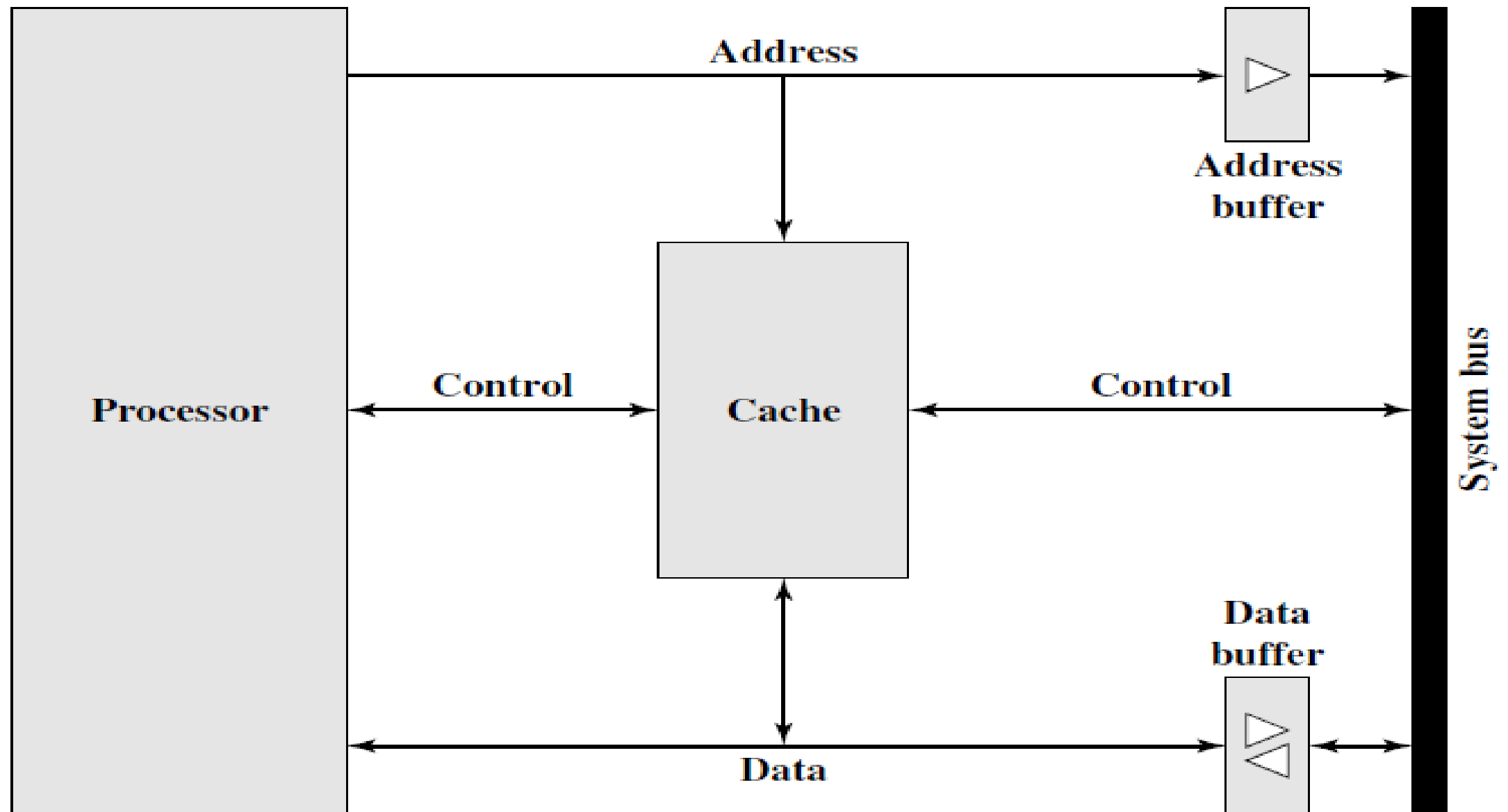


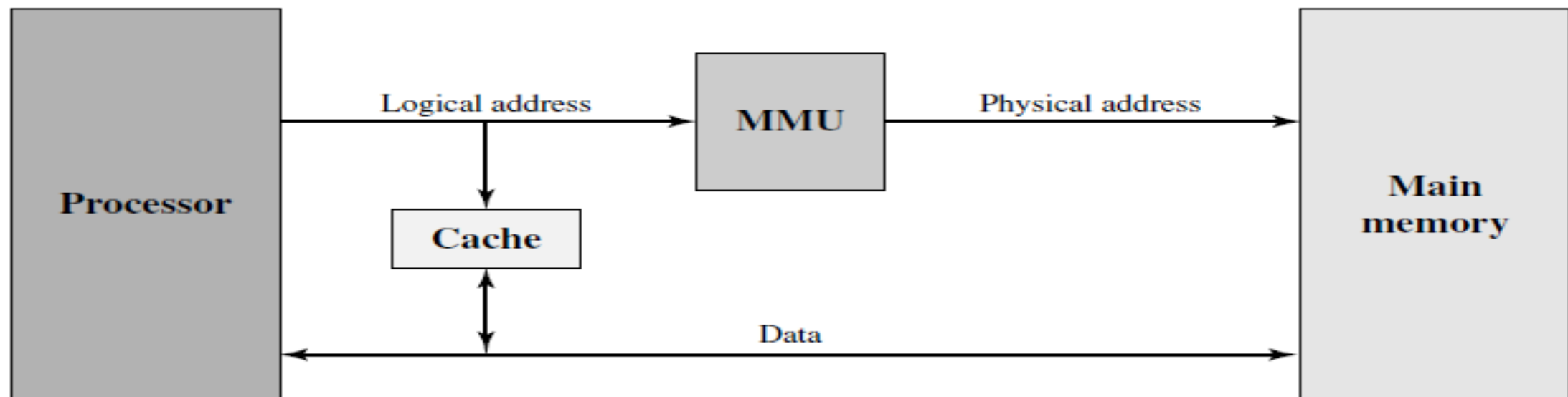
Figure 4.6 Typical Cache Organization

Table 4.2 Elements of Cache Design

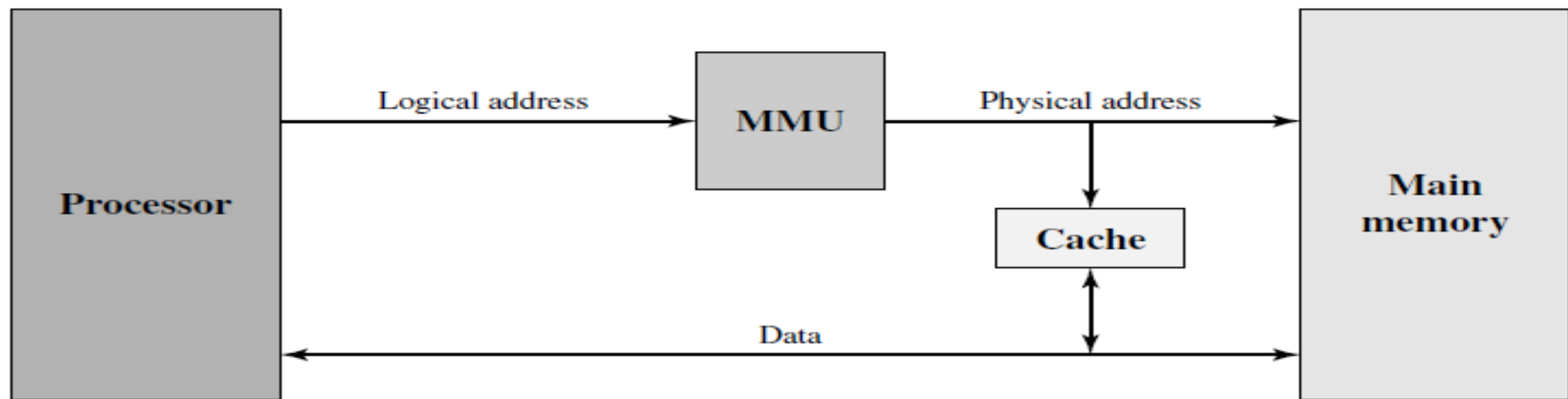
Cache Addresses	Write Policy
Logical	Write through
Physical	Write back
Cache Size	Write once
Mapping Function	Line Size
Direct	Number of caches
Associative	Single or two level
Set Associative	Unified or split
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

Cache elements

- When virtual addresses are used, the system designer may choose to place the cache between the processor and the MMU or between the MMU and main memory (Figure 4.7).
- A **logical cache**, also known as a **virtual cache**, stores data using **virtual addresses**. The processor accesses the cache directly, without going through the MMU. A physical cache stores data using main memory **physical addresses**.



(a) Logical cache



(b) Physical cache

Cache Size

- There are several other motivations for minimizing cache size.
- The larger the cache, the larger the number of gates involved in addressing the cache.
- The result is that large caches tend to be slightly slower than small ones—even when built with the same integrated circuit technology and put in the

Table 4.3 Cache Sizes of Some Processors

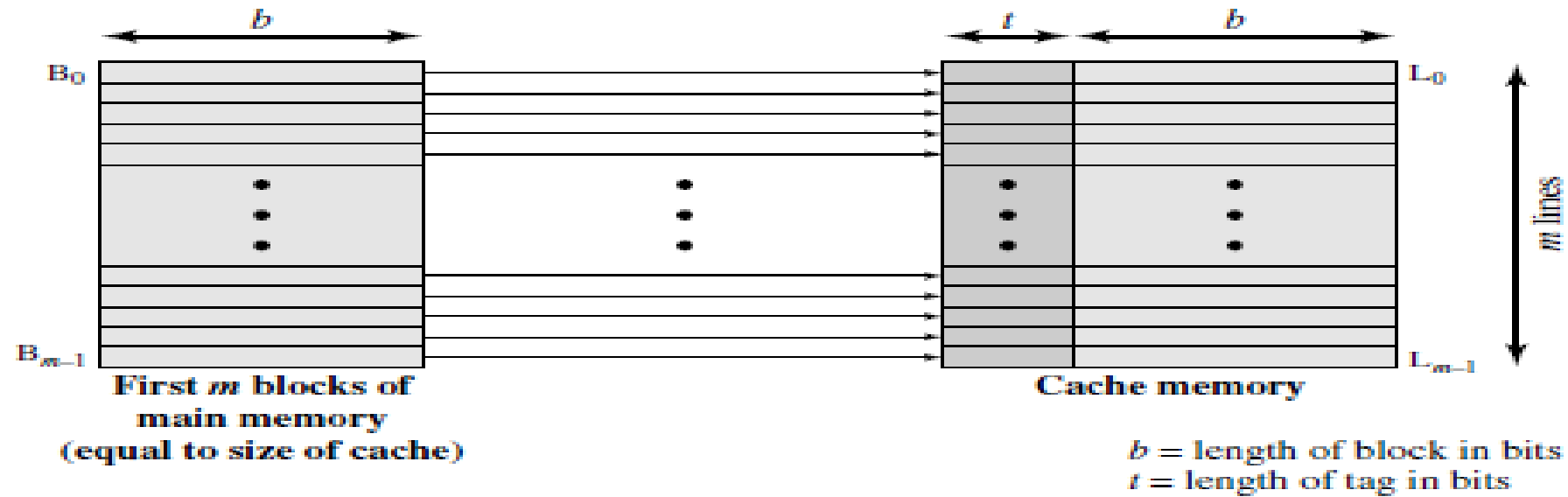
Processor	Type	Year of Introduction	L1 Cache ^a	L2 Cache	L3 Cache
IBM 360/85	Mainframe	1968	16 to 32 kB	—	—
PDP-11/70	Minicomputer	1975	1 kB	—	—
VAX 11/780	Minicomputer	1978	16 kB	—	—
IBM 3033	Mainframe	1978	64 kB	—	—
IBM 3090	Mainframe	1985	128 to 256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256 to 512 KB	—
PowerPC 601	PC	1993	32 kB	—	—
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/server	1999	32 kB/32 kB	256 KB to 1 MB	2 MB
IBM S/390 G4	Mainframe	1997	32 kB	256 KB	2 MB
IBM S/390 G6	Mainframe	1999	256 kB	8 MB	—
Pentium 4	PC/server	2000	8 kB/8 kB	256 KB	—
IBM SP	High-end server/ supercomputer	2000	64 kB/32 kB	8 MB	—
CRAY MTA ^b	Supercomputer	2000	8 kB	2 MB	—
Itanium	PC/server	2001	16 kB/16 kB	96 KB	4 MB
SGI Origin 2001	High-end server	2001	32 kB/32 kB	4 MB	—
Itanium 2	PC/server	2002	32 kB	256 KB	6 MB
IBM POWER5	High-end server	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 kB/64 kB	1 MB	—
IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24–48 MB

Mapping Function

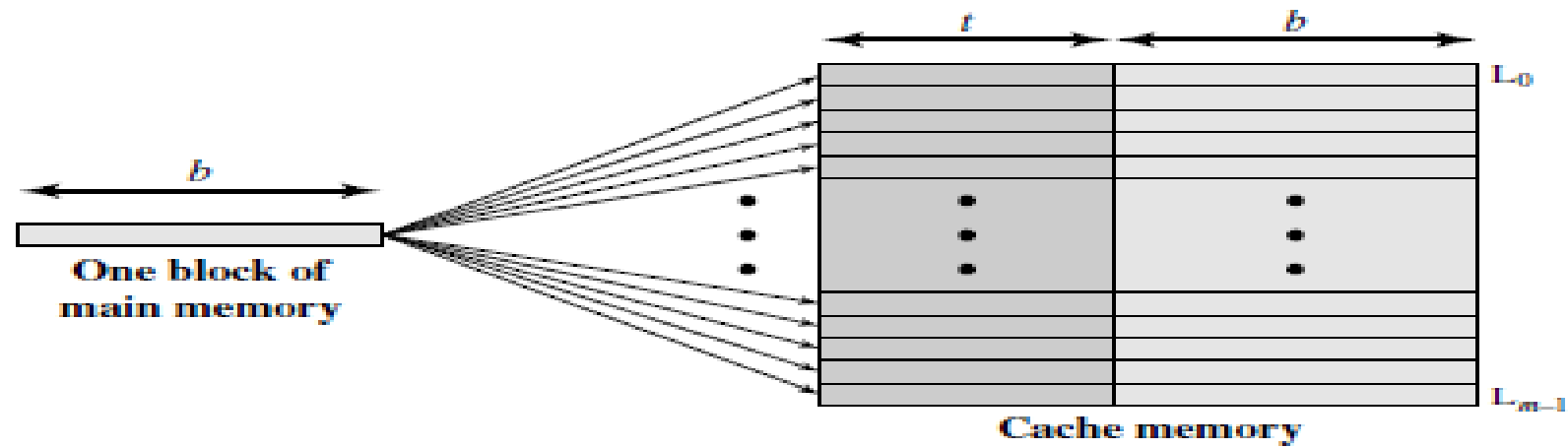
- Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines. Further, a means is needed for determining which main memory block currently occupies a cache line.
- The choice of the mapping function dictates how the cache is organized. Three techniques can be used: direct, associative, and set associative. We examine each of these in turn. In each case, we look at the general structure and then a specific example.

DIRECT MAPPING

- The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line. The mapping is expressed as where i cache line number j main memory block number m number of lines in the cache
- Figure 4.8a shows the mapping for the first blocks of main memory. Each block of main memory maps into one unique line of the cache. The next blocks of main memory map into the cache in the same fashion; that is, block B_m of main memory maps into line L_0 of cache, block B_{m+1} maps into line L_1 , and so on.



(a) Direct mapping



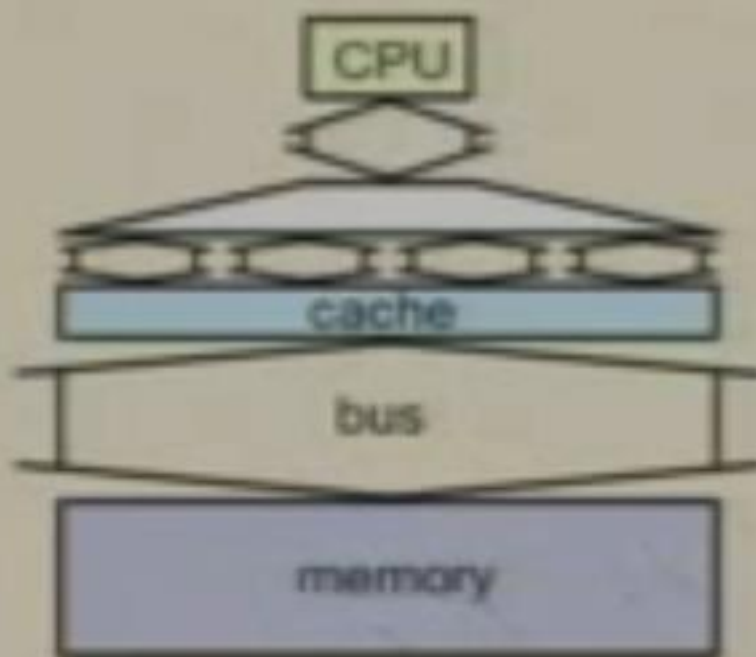
(b) Associative mapping

Figure 4.8 Mapping from Main Memory to Cache: Direct and Associative

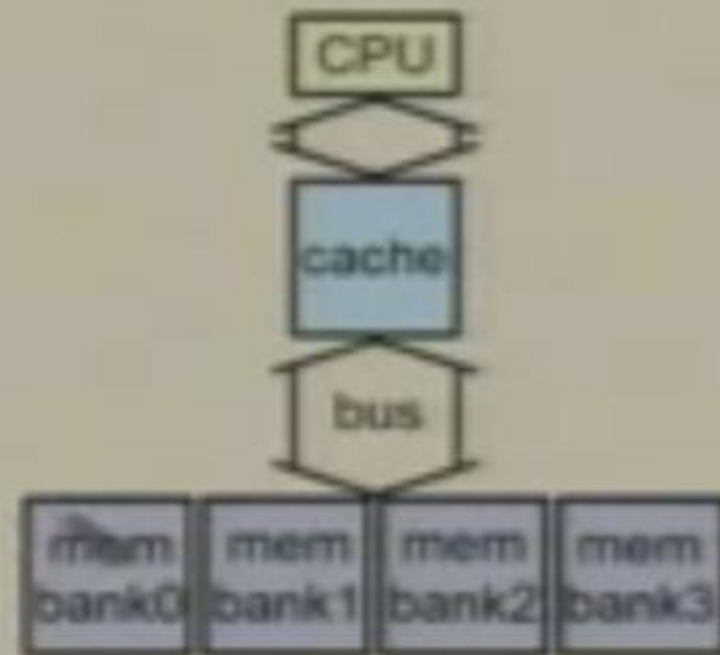
Transferring blocks to/from memory



a. one word wide memory

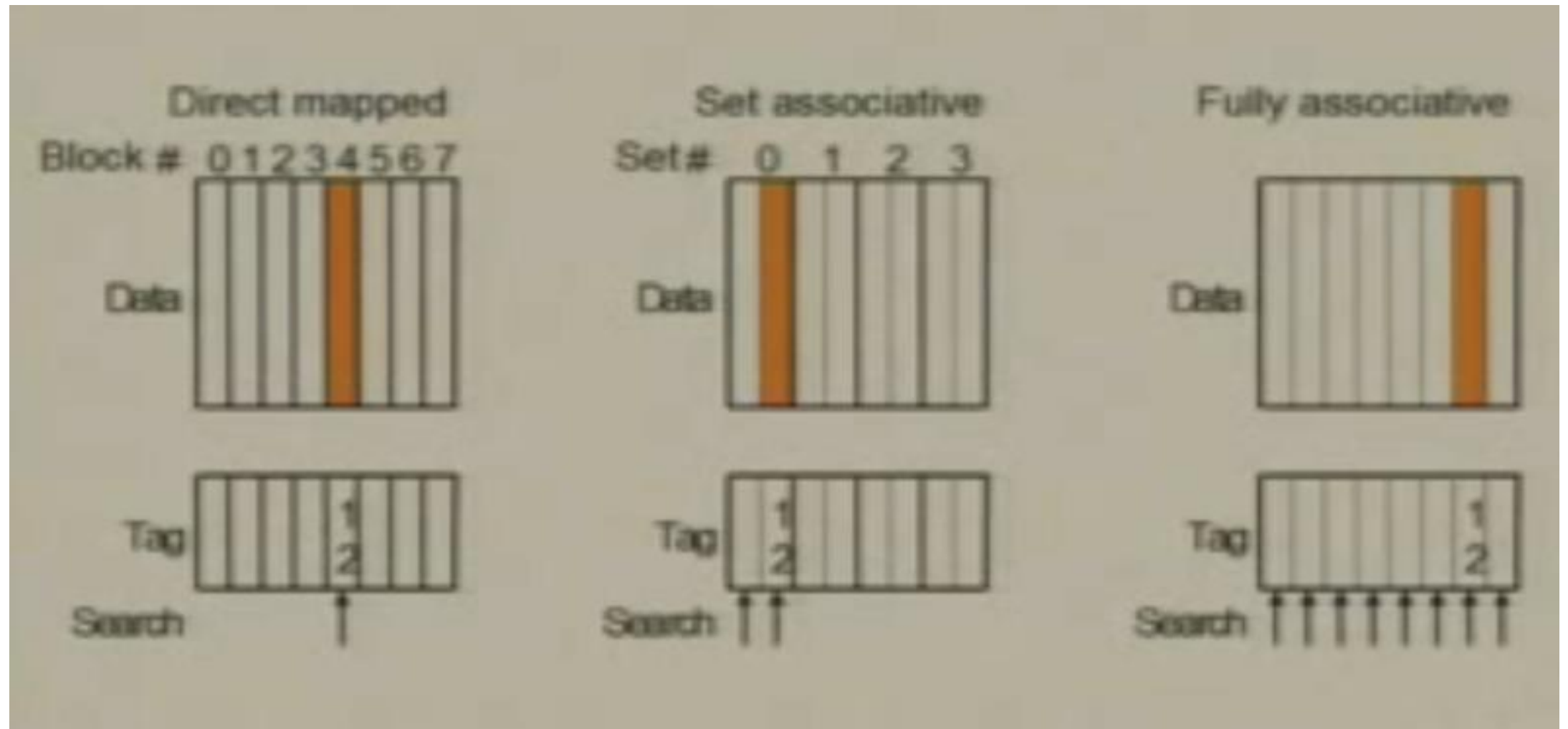


b. four word wide memory



c. interleaved memory

Cont.



Replacement Algorithms

- Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced.
- **For direct mapping, there is only one possible line for any particular block, and no choice is possible.** For the associative and set associative techniques, a replacement algorithm is needed. To achieve high speed, such an algorithm must be implemented in hardware.
- least recently used (LRU): Replace that block in the set that has been in the cache longest with no reference to it.
- Another possibility is first-in-first-out (FIFO): Replace that block in the set that has been in the cache longest.

Thanks for attentions